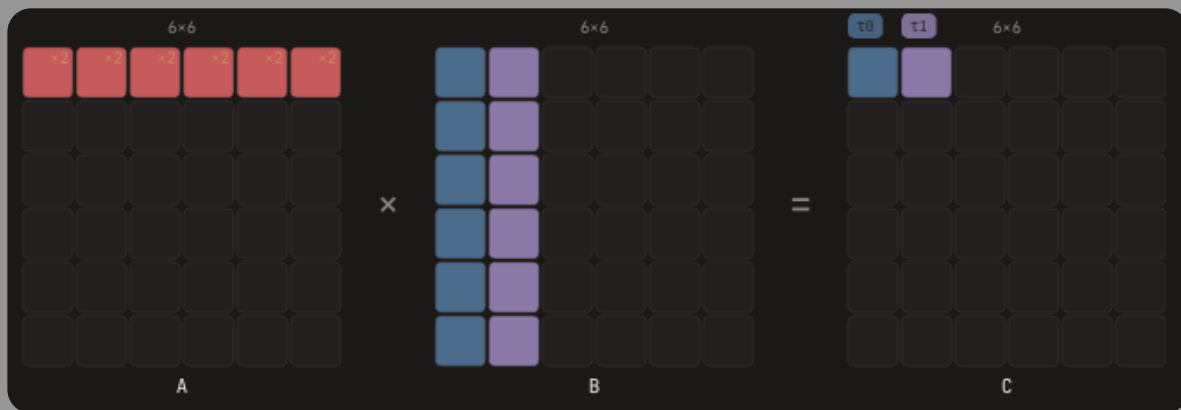


## → The Problem

### ⇒ Redundant Reads

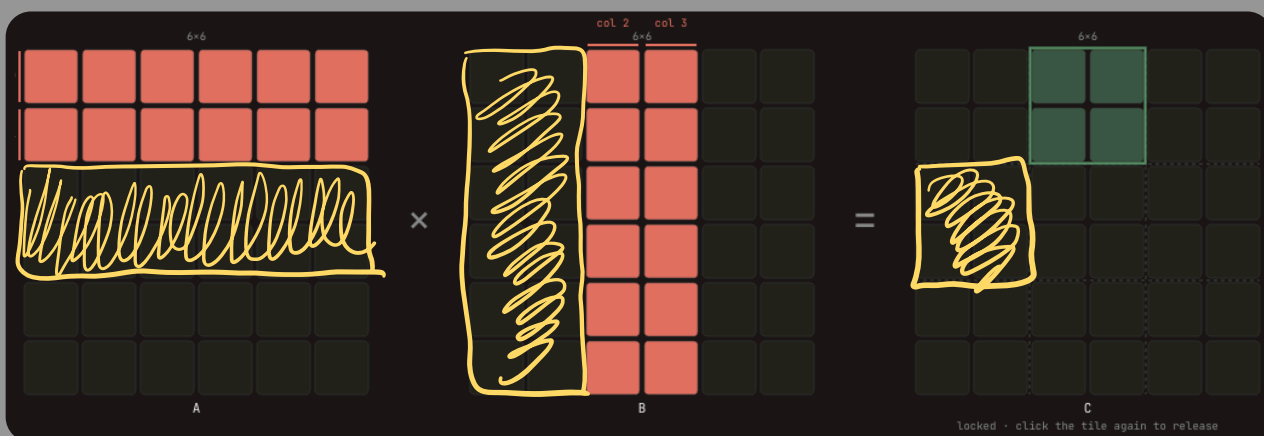


> both threads stream the same row of A from HBM,  
and every output cell does it again

Naive MatMul is  $O(N^3)$  reads at  
roughly 1 flop per byte:  
pure memory traffic.

## → How it works

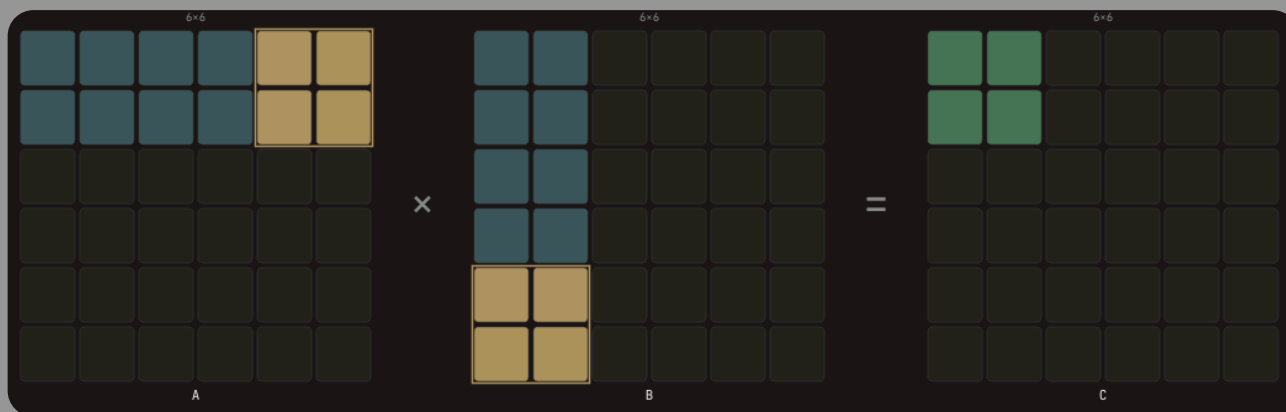
### ⇒ Explore Dependencies



each tile reads a fixed band of rows from A  
and cols from B;

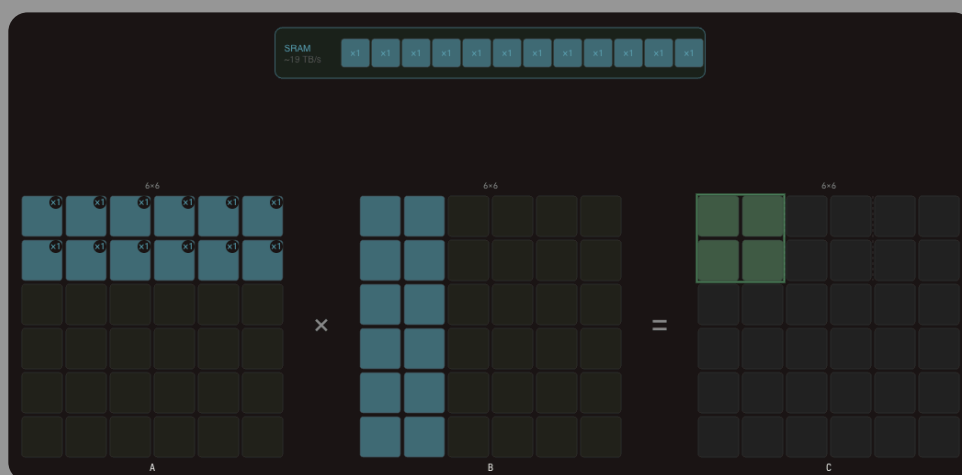
tiling loads that bond once into SRAM  
and reuses it  
for every cell in the tile

⇒ the Tiled Kernel



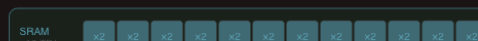
> The tiled kernel walk  $\times$  in phases!  
each phase loads A and B tiles  
from HBM into SRAM,  
multiplies on-chip,  
and accumulates into the output tile

⇒ Data Reuse

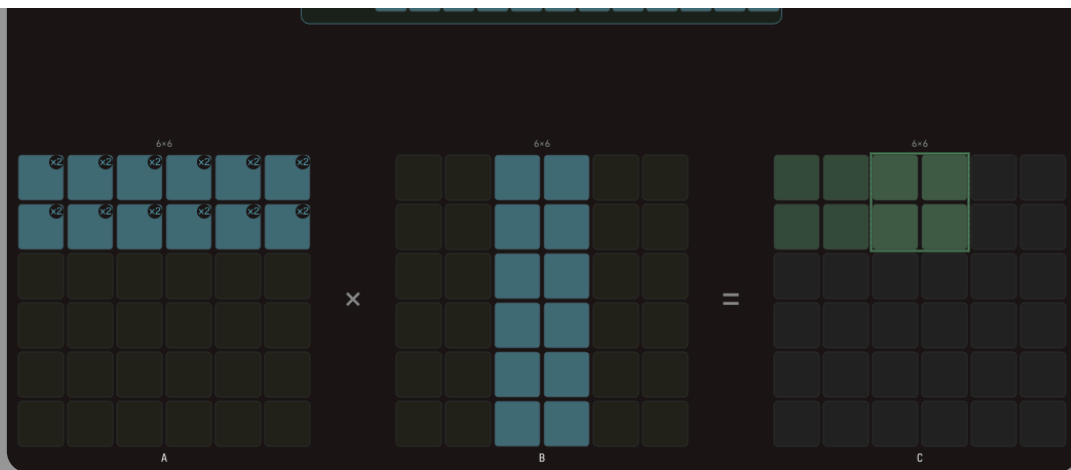


> compute  $c[0:2, 0:2]$   
both A and B tiles  
feed from SRAM at  $\sim 19\text{TB/s}$   
first output tile computed  
A row marked (x1)

> compute  $c[0:2, 2:4]$



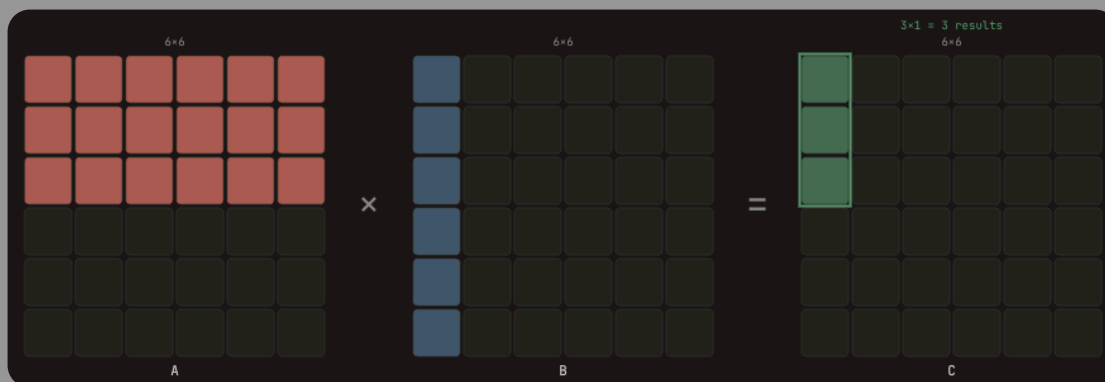
A rows reused from  
SRAM, zero HBM cost  
Second output tile  
computed  
A now  $\times 2$



> Load once into SRAM, compute many times  
Every value fetched from HBM is reused  
across multiple outputs;  
that's the bandwidth saving tiling provides

→ Block Tiling

⇒ 1D vs 2D



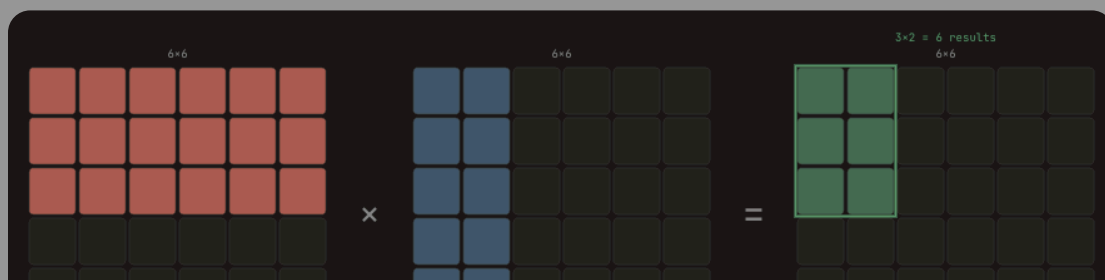
> TM = 3  
1D tile  
A loads: 18  
B loads: 6  
C loads: 6  
Per result: 10

↳ each thread computes TM outputs in a col.  
The B col. is shared across all TM results

> TM = 3 ; TN = 2

2D tile

A loads: 18  
B loads: 12



C loads: 12  
Per result: 7



↳ 2D blocktiling is how production GEMM  
kernels are structured:  
each thread owns a small rectangle of C  
and max. register reuse

This is the building block cuBLAS  
is built on